From “Selfish” to Optimal:

A Data-Driven Analysis of Shot Decision Quality Using Expected Goals

In football discussions, players are often labeled as “selfish” when they choose to shoot instead of passing. However, such judgments are often subjective and outcome-driven.

A missed shot does not necessarily imply a poor decision, and a goal does not always indicate an optimal one.

In this project, we propose a **data-driven framework** to evaluate shot decision quality by comparing:

- the expected goal value of the **actual shot**
- the expected goal value of a **reasonable passing alternative** from the same location under real match context.

Our goal is to examine whether player-level shooting decision quality can be quantified and compared using aggregated decision cost derived from match event data.

Project Context

The word **“selfish”** is commonly used in football to describe a player who chooses to shoot instead of passing when facing the goal. However, subjective judgments often fail to capture the true quality of decision-making.

A missed shot does not necessarily mean the decision was poor, and a goal does not always mean the decision was optimal. To evaluate whether a decision is truly “selfish”, we need to compare the actual shot with realistic passing alternatives under the same match situation.

This project investigates whether match event data and expected goals (**xG**) can be used to assess shooting decisions more objectively. Rather than judging a player only by the final outcome, we aim to evaluate whether the choice to shoot was the best available option under real match context.

Hypothesis

We hypothesize that:

1. Some shots that are commonly described as “selfish” are indeed lower-quality decisions, because passing could have led to a higher expected scoring opportunity.
2. Match context, such as time remaining, current scoreline, recent form, and league standing pressure, affects the quality of a shooting decision.
3. The same type of shot may be reasonable in one context but less optimal in another.
4. Player-level differences in shooting decision quality can be measured quantitatively using event data and aggregated decision cost.

Objective

The main objectives of this project are:

- to evaluate shooting decisions using expected goals rather than outcome alone;
- to compare the value of an actual shot with a realistic passing alternative;
- to incorporate match context into the assessment of decision quality;
- to quantify and compare player-level shooting decision quality using aggregated decision cost.

Data Sources Overview and Justified Source Selection

To study shot decision quality in Premier League matches, we combine **three public data sources**, each serving a different role in the pipeline. This combination was selected not simply for availability, but because no single public source provides all information required for our problem.

Source 1 — StatsBomb Open Data (Event-Level Match Data)

Why selected

- Provides detailed event-level information such as **shots, passes, xG, match time, and pitch location**
- Enables **decision-level analysis**, since our project compares the value of an actual shot with a plausible passing alternative
- Offers sufficiently high granularity for **feature construction**, including zone-based shot/pass linkage

Why it is necessary for our problem Our core research question is about whether a player’s decision to shoot was good relative to a passing alternative. This cannot be studied using only match-level results. We need event-level shot and pass data, so StatsBomb is the essential primary source.

Source 2 — Football-Data.co.uk (Match Results and Historical Records)

Why selected

- Provides complete and continuous match-result records across many seasons
- Supplies reliable match dates and score outcomes for temporal alignment
- Complements the more limited public coverage of event-level data

Why it is necessary for our problem We use this source to compute **recent form** and to support broader match-level context preparation. This is important because decision quality may depend not only on the shot itself, but also on the competitive context surrounding the match.

Source 3 — League Standings / Table Context

Why selected

- Provides team-level contextual information such as **points and league position**
- Helps approximate **match pressure** and competitive importance
- Supports contextual feature engineering beyond pure event data

Why it is necessary for our problem Shot decisions are not made in a vacuum. A shot taken by a team under pressure near the end of a season may differ in meaning from a similar shot taken in a low-pressure context. League standings therefore provide a useful contextual layer.

Why these three sources were combined

We selected this combination because it gives us:

1. **event-level decision data** (StatsBomb),
2. **complete historical match records** (Football-Data.co.uk), and
3. **competitive context** (league standings).

This makes the source selection aligned with the project goal rather than merely data availability.

Known limitations and accepted trade-offs

- StatsBomb public EPL event coverage is more limited than the full results window
- Public data does not include full tracking information such as defender positioning or passing lanes
- League standings and recent form are contextual proxies rather than direct measurements of psychological pressure

Despite these limitations, the chosen sources are sufficient to build a transparent and reproducible public-data pipeline for evaluating shot decision quality.

Lab 2 Focus: Source Loading and Preparation Pipeline

After justifying the choice of data sources, the next step is to load each source and prepare them for integration.

For Lab 2, the key focus is not only whether the sources exist, but whether they can be:

- loaded reproducibly,
- standardized consistently,
- integrated reliably,
- and assessed for resulting data quality.

The following sections therefore document how each selected source is prepared for downstream use.

Main Purpose: Fetch and inspect the Premier League competition list and the raw StatsBomb schema **What It Does:** Imports pandas/numpy and a single `get_json` helper built on the standard library; loads the competitions metadata; filters the Premier League; keeps the ten most recent EPL seasons; downloads one season's match list and one match's events so the nested JSON structure is visible before flattening **Benefits:** Establishes the foundation for event-level analysis without extra HTTP dependencies; documents the raw schema in one reusable place; every later request reuses `get_json`

```
import json
import urllib.request
import warnings

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

BASE = "https://raw.githubusercontent.com/statsbomb/open-data/master/data"

# football-data.co.uk mixes dd/mm/yy and dd/mm/yyyy, so dayfirst inference is required
warnings.filterwarnings("ignore", message="Could not infer format")


def get_json(url):
    """Download and parse one JSON document."""
    with urllib.request.urlopen(url) as response:
        return json.load(response)


competitions = pd.DataFrame(get_json(f"{BASE}/competitions.json"))
epl = (
    competitions[competitions["competition_name"] == "Premier League"]
    .sort_values("season_name", ascending=False)
    .head(10)
    .reset_index(drop=True)
)

print("Most recent 10 Premier League seasons available in StatsBomb:")
display(epl[["competition_id", "season_id", "season_name"]])

demo = epl.iloc[0]
demo_matches = pd.DataFrame(
    get_json(f"{BASE}/matches/{int(demo['competition_id'])}/{int(demo['season_id'])}.json")
)
print(f"Demo season selected for event preview: {demo['season_name']}")
print("Matches shape:", demo_matches.shape)
display(demo_matches[["match_id", "match_date", "home_team", "away_team"]].head())

demo_match_id = int(demo_matches.iloc[0]["match_id"])
demo_events = pd.DataFrame(get_json(f"{BASE}/events/{demo_match_id}.json"))
print(f"Loaded events for match_id = {demo_match_id}")
print("Events shape:", demo_events.shape)
display(demo_events[["id", "index", "minute", "second", "type", "possession"]].head())
```

```
Most recent 10 Premier League seasons available in StatsBomb:
Demo season selected for event preview: 2015/2016
Matches shape: (380, 18)
Loaded events for match_id = 3754217
Events shape: (3732, 38)
```

	competition_id	season_id	season_name
0	2	27	2015/2016
1	2	44	2003/2004

	match_id	match_date	home_team	away_team
0	3754217	2015-09-19	{'home_team_id': 33, 'home_team_name': 'Chelsea', 'home_team	{'away_team_id': 1, 'away_team_name': 'Arsenal', 'away_team_
1	3754117	2015-12-13	{'home_team_id': 59, 'home_team_name': 'Aston Villa', 'home_	{'away_team_id': 1, 'away_team_name': 'Arsenal', 'away_team_
2	3754296	2015-12-21	{'home_team_id': 1, 'home_team_name': 'Arsenal', 'home_team_	{'away_team_id': 36, 'away_team_name': 'Manchester City', 'a
3	3753983	2015-10-31	{'home_team_id': 26, 'home_team_name': 'Swansea City', 'home	{'away_team_id': 1, 'away_team_name': 'Arsenal', 'away_team_
4	3754160	2015-12-05	{'home_team_id': 1, 'home_team_name': 'Arsenal', 'home_team_	{'away_team_id': 41, 'away_team_name': 'Sunderland', 'away_t

	id	index	minute	second	type	possession
0	9d86a178-3514-45d1-9d14-1372e846d17b	1	0	0	{'id': 35, 'name': 'Starting XI'}	1
1	82acc213-90f3-4fee-8305-bd9e403cec42	2	0	0	{'id': 35, 'name': 'Starting XI'}	1
2	d18f1a33-65ba-42e5-a9fd-bbfc709694e8	3	0	0	{'id': 18, 'name': 'Half Start'}	1
3	49147091-455c-4372-8d56-96e1ca71d01a	4	0	0	{'id': 18, 'name': 'Half Start'}	1
4	5f9bdb6d-0379-4f42-9f53-29777f166db5	5	0	0	{'id': 30, 'name': 'Pass'}	2

We use football-data.co.uk to obtain:

- match results
- goals for / against
- match dates

This allows us to compute recent form (last 5 matches) for each team.

Football-Data.co.uk: Match-Level Historical Records

This source is used as the match-level backbone for temporal context construction.

In the Lab 2 scope, its main role is to support:

- reliable match dates and match outcomes,
- construction of recent-form features,
- alignment of event-level observations with broader match context.

This source complements the event-level source because event data alone is not sufficient for continuous historical context across seasons.

Main Purpose: Load ten seasons of Premier League match results and scores from football-data.co.uk **What It Does:** Loops over the ten season codes (2014-15 to 2023-24), downloads each season's CSV, tags it with its season code, concatenates the seasons and keeps the seven analysis columns; parses dates, coerces the goal columns to integers and drops incomplete records **Benefits:** Centralises match results in one consistently typed table; provides complete match outcomes (win/draw/loss and goals) used for league position and recent form

```
SEASON_CODES = [
    "1415", "1516", "1617", "1718", "1819",
    "1920", "2021", "2122", "2223", "2324",
]

results = pd.concat(
    [
        pd.read_csv(f"https://www.football-data.co.uk/mmz4281/{code}/E0.csv").assign(
            season_code=code
        )
        for code in SEASON_CODES
    ],
    ignore_index=True,
)[["season_code", "Date", "HomeTeam", "AwayTeam", "FTHG", "FTAG", "FTR"]]

results["Date"] = pd.to_datetime(results["Date"], dayfirst=True, errors="coerce")
results[["FTHG", "FTAG"]] = results[["FTHG", "FTAG"]].apply(pd.to_numeric, errors="coerce")
results = results.dropna(
    subset=["season_code", "Date", "HomeTeam", "AwayTeam", "FTHG", "FTAG", "FTR"]
).copy()
results[["FTHG", "FTAG"]] = results[["FTHG", "FTAG"]].astype(int)

display(results.head())
```

	season_code	Date	HomeTeam	AwayTeam	FTHG	FTAG	FTR
0	1415	2014-08-16	Arsenal	Crystal Palace	2	1	H
1	1415	2014-08-16	Leicester	Everton	2	2	D
2	1415	2014-08-16	Man United	Swansea	1	2	A
3	1415	2014-08-16	QPR	Hull	0	1	A
4	1415	2014-08-16	Stoke	Aston Villa	0	1	A

League-Table Context: Competitive Pressure Proxy

League-table information is introduced as a contextual layer rather than a primary event source.

Its role in the preparation pipeline is to provide:

- team ranking,
- accumulated points,
- season-stage competitive context.

We use this information as a proxy for match pressure and contextual importance. This is not a direct measurement of psychological pressure, but it provides a transparent and reproducible public-data approximation.

Main Purpose: Build one team-by-match table and reconstruct the league table after every match date **What It Does:** Reshapes each match into two team rows (home and away) with goals for/against and points; derives cumulative points, goal difference and games played per team-season; then, for every match date, keeps each team's latest cumulative record and ranks all teams by points, goal difference, goals for and name **Benefits:** One table serves both the recent-form feature and the league-position feature, so points are computed once; the rank ordering is deterministic and reproduces real league standings after each round

```
def build_team_match_table(results):
    """One row per team per match with goals, points and cumulative record."""
    home = pd.DataFrame({
        "season_code": results["season_code"],
        "Date": results["Date"],
        "team": results["HomeTeam"],
        "opponent": results["AwayTeam"],
        "is_home": True,
        "goals_for_match": results["FTHG"],
        "goals_against_match": results["FTAG"],
        "match_points": np.select([results["FTR"] == "H", results["FTR"] == "D"], [3, 1], 0),
    })
    away = pd.DataFrame({
        "season_code": results["season_code"],
        "Date": results["Date"],
        "team": results["AwayTeam"],
        "opponent": results["HomeTeam"],
        "is_home": False,
        "goals_for_match": results["FTAG"],
        "goals_against_match": results["FTHG"],
        "match_points": np.select([results["FTR"] == "A", results["FTR"] == "D"], [3, 1], 0),
    })

    rows = pd.concat([home, away], ignore_index=True)
    rows = rows.sort_values(["season_code", "team", "Date"]).reset_index(drop=True)

    per_team = rows.groupby(["season_code", "team"], sort=False)
    rows["games_played"] = per_team.cumcount() + 1
    rows["cum_points"] = per_team["match_points"].cumsum()
    rows["cum_gf"] = per_team["goals_for_match"].cumsum()
    rows["cum_ga"] = per_team["goals_against_match"].cumsum()
    rows["cum_gd"] = rows["cum_gf"] - rows["cum_ga"]
    return rows


def build_league_table(team_rows):
    """League position for every team after every match date of a season."""
    snapshots = []
    for season_code, season_rows in team_rows.groupby("season_code", sort=True):
        wide = (
            season_rows
            .pivot_table(
                index="Date",
                columns="team",
                values=["cum_points", "cum_gd", "cum_gf"],
                aggfunc="last",
            )
            .sort_index()
            .ffill()
        )
        teams = wide["cum_points"].columns.to_numpy()
        points, goal_diff, goals_for = (
            wide["cum_points"].to_numpy(),
            wide["cum_gd"].to_numpy(),
            wide["cum_gf"].to_numpy(),
        )

        for i, date in enumerate(wide.index):
            played = ~np.isnan(points[i])
            # ranking rule: points, then goal difference, then goals for, then name
            order = np.lexsort(
                (teams[played], -goals_for[i][played], -goal_diff[i][played], -points[i][played])
            )
            snapshots.append(pd.DataFrame({
                "season_code": season_code,
                "ranking_date": date,
                "team": teams[played][order],
                "cum_points": points[i][played][order],
                "cum_gd": goal_diff[i][played][order],
                "cum_gf": goals_for[i][played][order],
                "league_position": np.arange(1, played.sum() + 1),
            }))
    return pd.concat(snapshots, ignore_index=True)


league_points = build_team_match_table(results)
print("league_points shape:", league_points.shape)
display(league_points.head(10))

league_table_daily = build_league_table(league_points)
print("league_table_daily shape:", league_table_daily.shape)
display(league_table_daily.head(20))
```

```
league_points shape: (7600, 13)
league_table_daily shape: (22303, 7)
```

	season_code	Date	team	opponent	is_home	goals_for_match	goals_against_match	match_points	games_played	cum_points	cum_gf	cum_ga	cum_gd
0	1415	2014-08-16	Arsenal	Crystal Palace	True	2	1	3	1	3	2	1	1
1	1415	2014-08-23	Arsenal	Everton	False	2	2	1	2	4	4	3	1
2	1415	2014-08-31	Arsenal	Leicester	False	1	1	1	3	5	5	4	1
3	1415	2014-09-13	Arsenal	Man City	True	2	2	1	4	6	7	6	1
4	1415	2014-09-20	Arsenal	Aston Villa	False	3	0	3	5	9	10	6	4
5	1415	2014-09-27	Arsenal	Tottenham	True	1	1	1	6	10	11	7	4
6	1415	2014-10-05	Arsenal	Chelsea	False	0	2	0	7	10	11	9	2
7	1415	2014-10-18	Arsenal	Hull	True	2	2	1	8	11	13	11	2
8	1415	2014-10-25	Arsenal	Sunderland	False	2	0	3	9	14	15	11	4
9	1415	2014-11-01	Arsenal	Burnley	True	3	0	3	10	17	18	11	7

	season_code	ranking_date	team	cum_points	cum_gd	cum_gf	league_position
0	1415	2014-08-16	Arsenal	3.0	1.0	2.0	1
1	1415	2014-08-16	Swansea	3.0	1.0	2.0	2
2	1415	2014-08-16	Aston Villa	3.0	1.0	1.0	3
3	1415	2014-08-16	Hull	3.0	1.0	1.0	4
4	1415	2014-08-16	Tottenham	3.0	1.0	1.0	5
5	1415	2014-08-16	Everton	1.0	0.0	2.0	6
6	1415	2014-08-16	Leicester	1.0	0.0	2.0	7
7	1415	2014-08-16	Sunderland	1.0	0.0	2.0	8
8	1415	2014-08-16	West Brom	1.0	0.0	2.0	9
9	1415	2014-08-16	Crystal Palace	0.0	-1.0	1.0	10
10	1415	2014-08-16	Man United	0.0	-1.0	1.0	11
11	1415	2014-08-16	QPR	0.0	-1.0	0.0	12
12	1415	2014-08-16	Stoke	0.0	-1.0	0.0	13
13	1415	2014-08-16	West Ham	0.0	-1.0	0.0	14
14	1415	2014-08-17	Man City	3.0	2.0	2.0	1
15	1415	2014-08-17	Arsenal	3.0	1.0	2.0	2
16	1415	2014-08-17	Liverpool	3.0	1.0	2.0	3
17	1415	2014-08-17	Swansea	3.0	1.0	2.0	4
18	1415	2014-08-17	Aston Villa	3.0	1.0	1.0	5
19	1415	2014-08-17	Hull	3.0	1.0	1.0	6

Data Integration Strategy

Because this project combines event-level data, match-level records, and contextual team information, a clear integration strategy is necessary.

Core join dimensions

The main dimensions used to align sources are:

- season,
- match date,
- home team,
- away team,
- and standardized team identifiers where needed.

Standardisation before merging

Before joining, team names are harmonized across sources to resolve naming inconsistencies. This step is essential because public football datasets often use slightly different naming conventions.

Join design

The integration pipeline uses:

- exact joins for match-level metadata where identifiers are directly available,
- and time-aware joins for contextual variables such as recent form or league standing.

Why this matters

This design reduces mismatches, improves reproducibility, and helps prevent look-ahead bias when attaching context to shot events.

Method Summary

For each shot:

1. Obtain xG_shot from StatsBomb
2. Estimate xG_pass from historical pass-to-shot patterns

3. Compute:

Δ = xG_pass – xG_shot

4. Introduce context weight x:

- match time
- score difference
- league position
- recent form

Decision Cost = $\Delta \times x$

We then aggregate Decision Cost at player level to evaluate systematic decision-making quality.

Main Purpose: Profile the ten-season results dataset before it is used **What It Does:** Reports shape, columns and dtypes; counts missing values; lists the distinct team names; counts matches per season; summarises the goal distributions **Benefits:** Provides data quality assurance, documents coverage, and surfaces identifier problems (for example club-name variants) before any join is attempted

```
print("results shape:", results.shape)

print("\nColumns:")
print(results.columns.tolist())

print("\nDtypes:")
display(results.dtypes)

print("\nMissing values:")
display(results.isna().sum().sort_values(ascending=False).to_frame("missing_count"))

all_teams = pd.unique(results[["HomeTeam", "AwayTeam"]].values.ravel())
print("\nDistinct teams across 10 seasons:")
print("Number of unique team names:", len(all_teams))
display(pd.Series(sorted(all_teams), name="team").head(40))

print("\nSeason counts:")
display(results["season_code"].value_counts().sort_index().to_frame("num_matches"))

print("\nBasic score summary:")
display(results[["FTHG", "FTAG"]].describe())
```

```
results shape: (3800, 7)

Columns:
['season_code', 'Date', 'HomeTeam', 'AwayTeam', 'FTHG', 'FTAG', 'FTR']

Dtypes:

Missing values:

Distinct teams across 10 seasons:
Number of unique team names: 34

Season counts:

Basic score summary:
```

	0
season_code	str
Date	datetime64[us]
HomeTeam	str
AwayTeam	str
FTHG	int64
FTAG	int64
FTR	str

	missing_count
season_code	0
Date	0
HomeTeam	0
AwayTeam	0
FTHG	0
FTAG	0
FTR	0

	team
0	Arsenal
1	Aston Villa
2	Bournemouth
3	Brentford
4	Brighton

	team
5	Burnley
6	Cardiff
7	Chelsea
8	Crystal Palace
9	Everton
...	...
24	Sheffield United
25	Southampton
26	Stoke
27	Sunderland
28	Swansea
29	Tottenham
30	Watford
31	West Brom
32	West Ham
33	Wolves

	num_matches
season_code	
1415	380
1516	380
1617	380
1718	380
1819	380
1920	380
2021	380
2122	380
2223	380
2324	380

	FTHG	FTAG
count	3800.000000	3800.000000
mean	1.547895	1.245263
std	1.320389	1.202504
min	0.000000	0.000000
25%	1.000000	0.000000
50%	1.000000	1.000000
75%	2.000000	2.000000
max	9.000000	9.000000

Main Purpose: Load every available Premier League match from StatsBomb open data **What It Does:** Iterates over the ten EPL seasons, requests each season's match list, tags it with competition/season identifiers and concatenates the seasons **Benefits:** Produces the match spine that links event data to the football-data.co.uk results; the season loop reports each request so failures are visible

```
match_frames = []
for season in epl.itertuples(index=False):
    competition_id, season_id = int(season.competition_id), int(season.season_id)
    season_matches = pd.DataFrame(
        get_json(f"{BASE}/matches/{competition_id}/{season_id}.json")
    ).assign(
        competition_id=competition_id,
        season_id=season_id,
        season_name=season.season_name,
    )
    match_frames.append(season_matches)
    print(f"[OK] {season.season_name}: {season_matches.shape}")

matches_all = pd.concat(match_frames, ignore_index=True)
print("\nmatches_all shape:", matches_all.shape)
display(matches_all.head())
```

```
[OK] 2015/2016: (380, 21)
[OK] 2003/2004: (38, 21)

matches_all shape: (418, 21)
```

	match_id	match_date	kick_off	competition	season	home_team	away_team	home_score	away_score	match_status	...	last_updated	last_updated_360	metadata	match_week	competition_stage	stadium	referee	competition_id	season_id	season_name
0	3754217	2015-09-19	11:45:0.000	{'competition_id': 2, 'country_name': 'England', 'competitio	{'season_id': 27, 'season_name': '2015/2016'}	{'home_team_id': 33, 'home_team_name': 'Chelsea', 'home_team	{'away_team_id': 1, 'away_team_name': 'Arsenal', 'away_team_	2	0	available	...	2025-12-16T17:01:18.696515	2021-06-13T16:17:31.694	{'data_version': '1.1.0', 'shot_fidelity_version': '2', 'xy_	6	{'id': 1, 'name': 'Regular Season'}	{'id': 10, 'name': 'Stamford Bridge', 'country': {'id': 68,	{'id': 6, 'name': 'Mike Dean', 'country': {'id': 68, 'name':	2	27	2015/2016
1	3754117	2015-12-13	12:30:0.000	{'competition_id': 2, 'country_name': 'England', 'competitio	{'season_id': 27, 'season_name': '2015/2016'}	{'home_team_id': 59, 'home_team_name': 'Aston Villa', 'home_	{'away_team_id': 1, 'away_team_name': 'Arsenal', 'away_team_	0	2	available	...	2025-12-16T17:04:24.176407	2021-06-13T16:17:31.694	{'data_version': '1.1.0', 'shot_fidelity_version': '2', 'xy_	16	{'id': 1, 'name': 'Regular Season'}	{'id': 211, 'name': 'Villa Park', 'country': {'id': 68, 'nam	{'id': 12, 'name': 'Kevin Friend', 'country': {'id': 68, 'na	2	27	2015/2016
2	3754296	2015-12-21	19:00:0.000	{'competition_id': 2, 'country_name': 'England', 'competitio	{'season_id': 27, 'season_name': '2015/2016'}	{'home_team_id': 1, 'home_team_name': 'Arsenal', 'home_team_	{'away_team_id': 36, 'away_team_name': 'Manchester City', 'a	2	1	available	...	2025-12-17T11:59:35.059270	2021-06-13T16:17:31.694	{'data_version': '1.1.0', 'shot_fidelity_version': '2', 'xy_	17	{'id': 1, 'name': 'Regular Season'}	{'id': 3, 'name': 'Emirates Stadium', 'country': {'id': 68,	{'id': 5, 'name': 'Andre Marriner', 'country': {'id': 68, 'n	2	27	2015/2016
3	3753983	2015-10-31	14:00:0.000	{'competition_id': 2, 'country_name': 'England', 'competitio	{'season_id': 27, 'season_name': '2015/2016'}	{'home_team_id': 26, 'home_team_name': 'Swansea City', 'home	{'away_team_id': 1, 'away_team_name': 'Arsenal', 'away_team_	0	3	available	...	2025-12-16T17:02:24.849544	2021-06-13T16:17:31.694	{'data_version': '1.1.0', 'shot_fidelity_version': '2', 'xy_	11	{'id': 1, 'name': 'Regular Season'}	{'id': 45, 'name': 'Swansea.com Stadium', 'country': {'id':	{'id': 12, 'name': 'Kevin Friend', 'country': {'id': 68, 'na	2	27	2015/2016
4	3754160	2015-12-05	14:00:0.000	{'competition_id': 2, 'country_name': 'England', 'competitio	{'season_id': 27, 'season_name': '2015/2016'}	{'home_team_id': 1, 'home_team_name': 'Arsenal', 'home_team_	{'away_team_id': 41, 'away_team_name': 'Sunderland', 'away_t	3	1	available	...	2025-12-16T17:03:25.661537	2021-06-13T16:17:31.694	{'data_version': '1.1.0', 'shot_fidelity_version': '2', 'xy_	15	{'id': 1, 'name': 'Regular Season'}	{'id': 3, 'name': 'Emirates Stadium', 'country': {'id': 68,	{'id': 9, 'name': 'Robert Madley', 'country': {'id': 68, 'na	2	27	2015/2016

Main Purpose: Flatten the nested match metadata into a small, joinable table **What It Does:** Extracts the home and away team names from their nested objects, parses the match date and keeps the five columns needed downstream **Benefits:** Removes nested JSON from the analytical tables; gives a one-row-per-match dimension table used to attach date, season and venue context to every event

```
matches_flat = pd.DataFrame({
    "match_id": matches_all["match_id"].astype(int),
    "match_date": pd.to_datetime(matches_all["match_date"], errors="coerce"),
    "season_name": matches_all["season_name"],
    "home_team_name": matches_all["home_team"].map(
        lambda team: team.get("home_team_name") if isinstance(team, dict) else None
    ),
    "away_team_name": matches_all["away_team"].map(
        lambda team: team.get("away_team_name") if isinstance(team, dict) else None
    ),
}).drop_duplicates()

print("matches_flat shape:", matches_flat.shape)
display(matches_flat.head())
```

```
matches_flat shape: (418, 5)
```

	match_id	match_date	season_name	home_team_name	away_team_name
0	3754217	2015-09-19	2015/2016	Chelsea	Arsenal
1	3754117	2015-12-13	2015/2016	Aston Villa	Arsenal
2	3754296	2015-12-21	2015/2016	Arsenal	Manchester City
3	3753983	2015-10-31	2015/2016	Swansea City	Arsenal
4	3754160	2015-12-05	2015/2016	Arsenal	Sunderland

Main Purpose: Load the play-by-play events for all matches and keep only the fields the analysis uses **What It Does:** Defines `flatten_events`, which pulls the event type, team, player, possession, coordinates and the pass/shot payloads out of each

nested record; downloads the 418 matches once and concatenates the flattened frames **Benefits:** A single download pass replaces the repeated full event load, and flattening at the point of download keeps memory proportional to the 17 fields actually used instead of every nested attribute in the raw feed

```
EVENT_COLUMNS = [
    "type", "team", "player", "possession_team", "play_pattern", "location", "pass", "shot",
]


def flatten_events(events, match_id):
    """Reduce one match's nested event JSON to the flat analysis fields."""
    events = events.copy()
    for column in EVENT_COLUMNS:
        if column not in events:
            events[column] = None

    def name(obj, key="name"):
        return obj.get(key) if isinstance(obj, dict) else None

    def coordinate(point, axis):
        return point[axis] if isinstance(point, list) and len(point) >= 2 else np.nan

    return pd.DataFrame({
        "match_id": match_id,
        "index": events["index"],
        "minute": events["minute"],
        "second": events["second"],
        "possession": events["possession"],
        "type_name": events["type"].map(name),
        "team_name": events["team"].map(name),
        "player_name": events["player"].map(name),
        "possession_team_name": events["possession_team"].map(name),
        "play_pattern_name": events["play_pattern"].map(name),
        "x": events["location"].map(lambda point: coordinate(point, 0)),
        "y": events["location"].map(lambda point: coordinate(point, 1)),
        "pass_recipient_name": events["pass"].map(
            lambda payload: name(payload.get("recipient")) if isinstance(payload, dict) else None
        ),
        "pass_end_x": events["pass"].map(
            lambda payload: coordinate(payload.get("end_location"), 0)
            if isinstance(payload, dict) else np.nan
        ),
        "pass_end_y": events["pass"].map(
            lambda payload: coordinate(payload.get("end_location"), 1)
            if isinstance(payload, dict) else np.nan
        ),
        "shot_xg": events["shot"].map(
            lambda payload: payload.get("statsbomb_xg") if isinstance(payload, dict) else np.nan
        ),
        "shot_outcome_name": events["shot"].map(
            lambda payload: name(payload.get("outcome")) if isinstance(payload, dict) else None
        ),
    })


event_frames = []
for i, match_id in enumerate(matches_flat["match_id"].dropna().astype(int).unique(), 1):
    event_frames.append(flatten_events(pd.DataFrame(get_json(f"{BASE}/events/{match_id}.json")), match_id))
    if i % 50 == 0:
        print(f"Loaded {i}/{matches_flat['match_id'].nunique()} matches")

events_all = pd.concat(event_frames, ignore_index=True)
print("\nevents_all shape:", events_all.shape)
display(events_all.head())
```

```
Loaded 50/418 matches
Loaded 100/418 matches
Loaded 150/418 matches
Loaded 200/418 matches
Loaded 250/418 matches
Loaded 300/418 matches
Loaded 350/418 matches
Loaded 400/418 matches

events_all shape: (1443174, 17)
```

	match_id	index	minute	second	possession	type_name	team_name	player_name	possession_team_name	play_pattern_name	x	y	pass_recipient_name	pass_end_x	pass_end_y	shot_xg	shot_outcome_name
0	3754217	1	0	0	1	Starting XI	Chelsea	NaN	Chelsea	Regular Play	NaN	NaN	NaN	NaN	NaN	NaN	NaN
1	3754217	2	0	0	1	Starting XI	Arsenal	NaN	Chelsea	Regular Play	NaN	NaN	NaN	NaN	NaN	NaN	NaN
2	3754217	3	0	0	1	Half Start	Chelsea	NaN	Chelsea	Regular Play	NaN	NaN	NaN	NaN	NaN	NaN	NaN
3	3754217	4	0	0	1	Half Start	Arsenal	NaN	Chelsea	Regular Play	NaN	NaN	NaN	NaN	NaN	NaN	NaN

	match_id	index	minute	second	possession	type_name	team_name	player_name	possession_team_name	play_pattern_name	x	y	pass_recipient_name	pass_end_x	pass_end_y	shot_xg	shot_outcome_name
4	3754217	5	0	0	2	Pass	Arsenal	Theo Walcott	Arsenal	From Kick Off	61.0	40.1	Mesut Özil	59.1	42.1	NaN	NaN

Main Purpose: Join match context onto the events and split out the shot and pass tables **What It Does:** Merges the flat events with the match dimension table; filters to shot and pass events; keeps the analysis columns; drops records without coordinates, xG, team or player **Benefits:** Produces the two focused tables the decision model needs, with complete key fields and no nested structures

```
events_flat = events_all.merge(matches_flat, on="match_id", how="left")

SHOT_COLUMNS = [
    "match_id", "season_name", "match_date", "home_team_name", "away_team_name",
    "index", "minute", "second", "team_name", "player_name",
    "possession", "possession_team_name", "x", "y", "shot_xg", "shot_outcome_name",
]
PASS_COLUMNS = [
    "match_id", "season_name", "match_date", "home_team_name", "away_team_name",
    "index", "minute", "second", "team_name", "player_name",
    "possession", "possession_team_name", "x", "y",
    "pass_end_x", "pass_end_y", "pass_recipient_name",
]

shots = events_flat.loc[events_flat["type_name"] == "Shot", SHOT_COLUMNS].dropna(
    subset=["x", "y", "shot_xg", "team_name", "player_name"]
).copy()
passes = events_flat.loc[events_flat["type_name"] == "Pass", PASS_COLUMNS].dropna(
    subset=["x", "y", "team_name", "player_name"]
).copy()

print("shots shape:", shots.shape)
print("passes shape:", passes.shape)
display(shots.head())
display(passes.head())
```

```
shots shape: (10837, 16)
passes shape: (404785, 17)
```

	match_id	season_name	match_date	home_team_name	away_team_name	index	minute	second	team_name	player_name	possession	possession_team_name	x	y	shot_xg	shot_outcome_name
95	3754217	2015/2016	2015-09-19	Chelsea	Arsenal	96	1	47	Arsenal	Aaron Ramsey	6	Arsenal	94.5	42.9	0.038832	Blocked
207	3754217	2015/2016	2015-09-19	Chelsea	Arsenal	208	5	57	Chelsea	Francesc Fàbregas i Soler	12	Chelsea	93.5	48.4	0.031541	Saved
442	3754217	2015/2016	2015-09-19	Chelsea	Arsenal	443	11	17	Arsenal	Alexis Alejandro Sánchez Sánchez	21	Arsenal	89.8	22.3	0.006660	Off T
909	3754217	2015/2016	2015-09-19	Chelsea	Arsenal	910	23	29	Chelsea	Diego da Silva Costa	42	Chelsea	98.2	26.0	0.022902	Saved
1096	3754217	2015/2016	2015-09-19	Chelsea	Arsenal	1097	28	36	Arsenal	Theo Walcott	50	Arsenal	105.7	24.2	0.049741	Saved

	match_id	season_name	match_date	home_team_name	away_team_name	index	minute	second	team_name	player_name	possession	possession_team_name	x	y	pass_end_x	pass_end_y	pass_recipient_name
4	3754217	2015/2016	2015-09-19	Chelsea	Arsenal	5	0	0	Arsenal	Theo Walcott	2	Arsenal	61.0	40.1	59.1	42.1	Mesut Özil
6	3754217	2015/2016	2015-09-19	Chelsea	Arsenal	7	0	1	Arsenal	Mesut Özil	2	Arsenal	59.3	42.1	43.7	38.2	Santiago Cazorla González
9	3754217	2015/2016	2015-09-19	Chelsea	Arsenal	10	0	5	Arsenal	Santiago Cazorla González	2	Arsenal	49.5	41.7	63.1	48.9	Aaron Ramsey
13	3754217	2015/2016	2015-09-19	Chelsea	Arsenal	14	0	7	Arsenal	Aaron Ramsey	2	Arsenal	65.9	48.7	69.6	48.9	Theo Walcott
18	3754217	2015/2016	2015-09-19	Chelsea	Arsenal	19	0	8	Chelsea	Francesc Fàbregas i Soler	3	Chelsea	47.1	38.2	32.6	39.3	Kurt Happy Zouma

Main Purpose: Assign every shot and pass to one of 16 pitch zones **What It Does:** Splits the 120 × 80 StatsBomb pitch into a 4 × 4 grid; labels each event `Zx_y` from its coordinates and leaves out-of-pitch coordinates unassigned **Benefits:** Provides the spatial unit that links a pass to the shot it eventually created; keeps the zone definition in one function applied to both tables

```
X_BINS = [0, 30, 60, 90, 120]
Y_BINS = [0, 20, 40, 60, 80]


def add_zone(frame):
    """Label each event with its 4 x 4 pitch zone."""
    x_bin = pd.cut(frame["x"], X_BINS, labels=False, include_lowest=True)
    y_bin = pd.cut(frame["y"], Y_BINS, labels=False, include_lowest=True)
    zone = "Z" + x_bin.astype("Int64").astype(str) + "_" + y_bin.astype("Int64").astype(str)
    return frame.assign(zone=zone.mask(x_bin.isna() | y_bin.isna()))


shots = add_zone(shots)
passes = add_zone(passes)

display(shots[["x", "y", "zone"]].head())
display(passes[["x", "y", "zone"]].head())
```

	x	y	zone
95	94.5	42.9	Z3_2
207	93.5	48.4	Z3_2
442	89.8	22.3	Z2_1
909	98.2	26.0	Z3_1
1096	105.7	24.2	Z3_1

	x	y	zone
4	61.0	40.1	Z2_2
6	59.3	42.1	Z1_2
9	49.5	41.7	Z1_2
13	65.9	48.7	Z2_2
18	47.1	38.2	Z1_1

Main Purpose: Estimate the value of the passing continuation available from each zone **What It Does:** Finds, for every pass, the next shot by the same team in the same possession; attaches that shot's xG; averages those next-shot xG values per pass zone to produce `xG_pass` with its sample size **Benefits:** Converts “what a pass could have led to” into a zone-level expected value that can be compared with the shot actually taken; the sample size documents how reliable each zone estimate is

```
shot_lookup = (
    shots[["match_id", "team_name", "possession", "index", "shot_xg"]]
    .sort_values("index")
)

pass_shot_df = (
    pd.merge_asof(
        passes.assign(_row=np.arange(len(passes))).sort_values("index"),
        shot_lookup,
        on="index",
        by=["match_id", "team_name", "possession"],
        direction="forward",
        allow_exact_matches=False,
    )
    .dropna(subset=["shot_xg"])
    .sort_values("_row")
    .rename(columns={"index": "pass_index", "shot_xg": "next_shot_xg"})
    [["match_id", "team_name", "possession", "pass_index", "zone", "next_shot_xg"]]
)

print("pass_shot_df shape:", pass_shot_df.shape)
display(pass_shot_df.head())

zone_xg_pass = (
    pass_shot_df
    .groupby("zone", as_index=False)
    .agg(
        xG_pass=("next_shot_xg", "mean"),
        zone_pass_shot_samples=("next_shot_xg", "size"),
    )
)

print("zone_xg_pass shape:", zone_xg_pass.shape)
display(zone_xg_pass.sort_values("zone"))
```

```
pass_shot_df shape: (52201, 6)
zone_xg_pass shape: (16, 3)
```

	match_id	team_name	possession	pass_index	zone	next_shot_xg
10894	3754217	Arsenal	6	91	Z3_0	0.038832
22217	3754217	Chelsea	12	186	Z2_3	0.031541
22492	3754217	Chelsea	12	189	Z2_2	0.031541
23169	3754217	Chelsea	12	194	Z2_2	0.031541
23492	3754217	Chelsea	12	197	Z2_2	0.031541

	zone	xG_pass	zone_pass_shot_samples
0	Z0_0	0.110314	595
1	Z0_1	0.097391	1436
2	Z0_2	0.101183	1458
3	Z0_3	0.095918	504
4	Z1_0	0.090557	3080
5	Z1_1	0.095169	3581
6	Z1_2	0.089113	3439
7	Z1_3	0.093408	2898
8	Z2_0	0.083987	5120
9	Z2_1	0.085757	5212
10	Z2_2	0.088106	4987
11	Z2_3	0.082572	5111
12	Z3_0	0.084102	4664
13	Z3_1	0.105515	2813

	zone	xG_pass	zone_pass_shot_samples
14	Z3_2	0.111455	2596
15	Z3_3	0.085231	4707

Main Purpose: Compare each shot's xG with the zone's historical passing value **What It Does:** Joins `xG_pass` and its sample size onto every shot; computes `delta = xG_pass − shot_xg` **Benefits:** Produces the per-shot expected-value gap that the decision-cost metric is built on

```
shots_enriched = shots.merge(zone_xg_pass, on="zone", how="left")
shots_enriched["delta"] = shots_enriched["xG_pass"] - shots_enriched["shot_xg"]

print("shots_enriched shape:", shots_enriched.shape)
display(shots_enriched[[
    "match_id", "player_name", "team_name", "zone",
    "shot_xg", "xG_pass", "zone_pass_shot_samples", "delta",
]].head(10))
```

```
shots_enriched shape: (10837, 20)
```

	match_id	player_name	team_name	zone	shot_xg	xG_pass	zone_pass_shot_samples	delta
0	3754217	Aaron Ramsey	Arsenal	Z3_2	0.038832	0.111455	2596.0	0.072623
1	3754217	Francesc Fàbregas i Soler	Chelsea	Z3_2	0.031541	0.111455	2596.0	0.079914
2	3754217	Alexis Alejandro Sánchez Sánchez	Arsenal	Z2_1	0.006660	0.085757	5212.0	0.079097
3	3754217	Diego da Silva Costa	Chelsea	Z3_1	0.022902	0.105515	2813.0	0.082614
4	3754217	Theo Walcott	Arsenal	Z3_1	0.049741	0.105515	2813.0	0.055775
5	3754217	Pedro Eliezer Rodríguez Ledesma	Chelsea	Z3_2	0.400975	0.111455	2596.0	-0.289520
6	3754217	Aaron Ramsey	Arsenal	Z3_1	0.023207	0.105515	2813.0	0.082309
7	3754217	Kurt Happy Zouma	Chelsea	Z2_2	0.020538	0.088106	4987.0	0.067568
8	3754217	Pedro Eliezer Rodríguez Ledesma	Chelsea	Z3_2	0.033888	0.111455	2596.0	0.077567
9	3754217	Francesc Fàbregas i Soler	Chelsea	Z3_1	0.026702	0.105515	2813.0	0.078813

Main Purpose: Reuse the team-by-match table to compute recent form **What It Does:** Derives the nine-column team-match view used for context joins and adds `recent_form_5` — points won in the previous five matches of the same season **Benefits:** Avoids rebuilding a second team-match table; the shifted rolling sum only uses information available before the match, so no look-ahead bias is introduced

```
team_matches = (
    league_points
    .rename(columns={"goals_for_match": "goals_for", "goals_against_match": "goals_against"})
    [["season_code", "Date", "team", "opponent", "is_home",
      "goals_for", "goals_against", "match_points"]]
)
team_matches["is_home"] = team_matches["is_home"].astype(int)
team_matches["recent_form_5"] = (
    team_matches
    .groupby(["season_code", "team"])["match_points"]
    .transform(lambda points: points.shift(1).rolling(5, min_periods=1).sum())
)

print("team_matches shape:", team_matches.shape)
display(team_matches.head(10))
```

```
team_matches shape: (7600, 9)
```

	season_code	Date	team	opponent	is_home	goals_for	goals_against	match_points	recent_form_5
0	1415	2014-08-16	Arsenal	Crystal Palace	1	2	1	3	NaN
1	1415	2014-08-23	Arsenal	Everton	0	2	2	1	3.0
2	1415	2014-08-31	Arsenal	Leicester	0	1	1	1	4.0
3	1415	2014-09-13	Arsenal	Man City	1	2	2	1	5.0
4	1415	2014-09-20	Arsenal	Aston Villa	0	3	0	3	6.0
5	1415	2014-09-27	Arsenal	Tottenham	1	1	1	1	9.0
6	1415	2014-10-05	Arsenal	Chelsea	0	0	2	0	7.0
7	1415	2014-10-18	Arsenal	Hull	1	2	2	1	6.0
8	1415	2014-10-25	Arsenal	Sunderland	0	2	0	3	6.0
9	1415	2014-11-01	Arsenal	Burnley	1	3	0	3	8.0

Contextual Feature Preparation

At this stage, the notebook moves from source-level cleaning to contextual feature preparation.

The goal is to enrich each event or match observation with information that would have been available at the relevant time, such as:

- recent team form,
- league position,
- broader competition context.

This step is still part of the Lab 2 scope because it concerns data preparation, integration, and management rather than final analytical interpretation.

Main Purpose: Attach recent form and league position to each shot with a time-aware join **What It Does:** Converts season names to football-data season codes and StatsBomb club names to their football-data spelling; keeps only seasons covered by both sources; then uses `merge_asof` per season and club to attach the latest context strictly before the shot's match date **Benefits:** A single `merge_asof` call replaces the hand-written grouped merge; `allow_exact_matches=False` guarantees the model only sees state from earlier matches, so the context features cannot leak the match being analysed

```
TEAM_JOIN_MAP = {
    "AFC Bournemouth": "Bournemouth",
    "Leicester City": "Leicester",
    "Manchester City": "Man City",
    "Manchester United": "Man United",
    "Newcastle United": "Newcastle",
    "Norwich City": "Norwich",
    "Stoke City": "Stoke",
    "Swansea City": "Swansea",
    "Tottenham Hotspur": "Tottenham",
    "West Bromwich Albion": "West Brom",
    "West Ham United": "West Ham",
    "Wolverhampton Wanderers": "Wolves",
    "Nottingham Forest": "Nott'm Forest",
}


def season_name_to_code(season_name):
    """'2015/2016' -> '1516', the season code used by football-data.co.uk."""
    parts = str(season_name).split("/") if pd.notna(season_name) else []
    return parts[0][-2:] + parts[1][-2:] if len(parts) == 2 else np.nan


def attach_context(shots_frame, context, value_columns):
    """Time-aware join: latest context row strictly before each shot's match date."""
    left = (
        shots_frame
        .assign(
            season_code=shots_frame["season_name"].map(season_name_to_code),
            team_join=shots_frame["team_name"].replace(TEAM_JOIN_MAP),
            _row=np.arange(len(shots_frame)),
        )
        .dropna(subset=["season_code", "team_join", "match_date"])
        .sort_values("match_date")
    )
    right = context.sort_values("Date")

    merged = pd.merge_asof(
        left,
        right[["season_code", "team_join", "Date", *value_columns]],
        left_on="match_date",
        right_on="Date",
        by=["season_code", "team_join"],
        direction="backward",
        allow_exact_matches=False,
    )
    return merged.sort_values("_row").drop(columns=["_row", "Date"]).reset_index(drop=True)


shots_overlap = shots_enriched[
    shots_enriched["season_name"].map(season_name_to_code).isin(results["season_code"].unique())
].copy()
print("StatsBomb seasons retained after overlap filtering:")
print(sorted(shots_overlap["season_name"].map(season_name_to_code).unique()))

shots_with_form = attach_context(
    shots_overlap,
    team_matches[["season_code", "Date", "team", "recent_form_5"]].rename(columns={"team": "team_join"}),
    ["recent_form_5"],
)
print("shots_with_form shape:", shots_with_form.shape)
display(shots_with_form[[
    "match_id", "player_name", "team_name", "season_code",
    "team_join", "match_date", "recent_form_5",
]].head(10))

shots_with_context = attach_context(
    shots_with_form,
    league_table_daily.rename(columns={"ranking_date": "Date", "team": "team_join"}),
    ["league_position", "cum_points", "cum_gd"],
)
print("shots_with_context shape:", shots_with_context.shape)
display(shots_with_context[[
    "match_id", "player_name", "team_name", "season_code",
    "team_join", "match_date", "recent_form_5",
    "league_position", "cum_points", "cum_gd",
]].head(10))
```

```
StatsBomb seasons retained after overlap filtering:
['1516']
shots_with_form shape: (9908, 23)
shots_with_context shape: (9908, 26)
```

	match_id	player_name	team_name	season_code	team_join	match_date	recent_form_5
0	3754217	Aaron Ramsey	Arsenal	1516	Arsenal	2015-09-19	7.0
1	3754217	Francesc Fàbregas i Soler	Chelsea	1516	Chelsea	2015-09-19	4.0
2	3754217	Alexis Alejandro Sánchez Sánchez	Arsenal	1516	Arsenal	2015-09-19	7.0
3	3754217	Diego da Silva Costa	Chelsea	1516	Chelsea	2015-09-19	4.0
4	3754217	Theo Walcott	Arsenal	1516	Arsenal	2015-09-19	7.0
5	3754217	Pedro Eliezer Rodríguez Ledesma	Chelsea	1516	Chelsea	2015-09-19	4.0
6	3754217	Aaron Ramsey	Arsenal	1516	Arsenal	2015-09-19	7.0
7	3754217	Kurt Happy Zouma	Chelsea	1516	Chelsea	2015-09-19	4.0
8	3754217	Pedro Eliezer Rodríguez Ledesma	Chelsea	1516	Chelsea	2015-09-19	4.0
9	3754217	Francesc Fàbregas i Soler	Chelsea	1516	Chelsea	2015-09-19	4.0

	match_id	player_name	team_name	season_code	team_join	match_date	recent_form_5	league_position	cum_points	cum_gd
0	3754217	Aaron Ramsey	Arsenal	1516	Arsenal	2015-09-19	7.0	4.0	10.0	2.0
1	3754217	Francesc Fàbregas i Soler	Chelsea	1516	Chelsea	2015-09-19	4.0	17.0	4.0	-5.0
2	3754217	Alexis Alejandro Sánchez Sánchez	Arsenal	1516	Arsenal	2015-09-19	7.0	4.0	10.0	2.0
3	3754217	Diego da Silva Costa	Chelsea	1516	Chelsea	2015-09-19	4.0	17.0	4.0	-5.0
4	3754217	Theo Walcott	Arsenal	1516	Arsenal	2015-09-19	7.0	4.0	10.0	2.0
5	3754217	Pedro Eliezer Rodríguez Ledesma	Chelsea	1516	Chelsea	2015-09-19	4.0	17.0	4.0	-5.0
6	3754217	Aaron Ramsey	Arsenal	1516	Arsenal	2015-09-19	7.0	4.0	10.0	2.0
7	3754217	Kurt Happy Zouma	Chelsea	1516	Chelsea	2015-09-19	4.0	17.0	4.0	-5.0
8	3754217	Pedro Eliezer Rodríguez Ledesma	Chelsea	1516	Chelsea	2015-09-19	4.0	17.0	4.0	-5.0
9	3754217	Francesc Fàbregas i Soler	Chelsea	1516	Chelsea	2015-09-19	4.0	17.0	4.0	-5.0

Merge Validation Summary

After integrating the different public sources, the key validation questions are:

- Were the expected rows preserved after merging?
- Which contextual fields were matched successfully?
- Are unmatched records explainable by source coverage gaps or missing historical context?

This validation is important because the reliability of any later analysis depends on whether the prepared dataset has been integrated consistently.

Main Purpose: Audit the cross-source merge before analysis **What It Does:** Reports row counts, contextual-column match rates, unmatched club names between the two sources, and duplicate checks on the shot key **Benefits:** Makes integration quality explicit; a silently unmatched club or a 100% missing context column is caught here instead of distorting the final metric

```
CONTEXT_COLUMNS = ["recent_form_5", "league_position", "cum_points", "cum_gd"]
KEY = ["match_id", "index"]

print("===== LAB 2 MERGE AUDIT =====")
print(f"Rows: {len(shots_with_context)}")
print(f"Columns: {shots_with_context.shape[1]}")

print("\nMatch rate of contextual columns:")
display(
    (1 - shots_with_context[CONTEXT_COLUMNS].isna().mean())
    .to_frame("match_rate")
    .round(3)
)

football_teams = set(pd.concat([results["HomeTeam"], results["AwayTeam"]]).dropna().unique())
statsbomb_teams = set(shots_with_context["team_join"].dropna().unique())
print("\nStatsBomb teams after normalization:", len(statsbomb_teams))
print("Unmatched teams:", sorted(statsbomb_teams - football_teams))

print("\nDuplicate shot keys:", shots_with_context.duplicated(subset=KEY).sum())
print("===== END OF AUDIT =====")
```

```
===== LAB 2 MERGE AUDIT =====
Rows: 9908
Columns: 26

Match rate of contextual columns:

StatsBomb teams after normalization: 20
Unmatched teams: []

Duplicate shot keys: 0
===== END OF AUDIT =====
```

	match_rate
recent_form_5	0.951

	match_rate
league_position	0.976
cum_points	0.976
cum_gd	0.976

Data Quality Summary for Lab 2

At the second-lab milestone, the main concern is not only whether data preparation has been completed, but also whether the resulting dataset is of sufficient quality for downstream analysis.

The main quality risks at this stage are:

- missing attributes in event-level data,
- incomplete contextual matches after cross-source integration,
- inconsistent identifiers across public sources,
- and small-sample instability in derived aggregates.

These aspects are checked explicitly in later sections, but they are highlighted here because they are central to the Lab 2 deliverable.

Beyond the Core Lab 2 Scope: Project-Specific Analytical Construction

Up to this point, the notebook has covered the main Lab 2 milestone components:

- justified source selection,
- source loading and standardisation,
- integration design,
- contextual data preparation,
- and data quality considerations.

The following sections go beyond the core Lab 2 focus. They show how the prepared dataset is used to construct project-specific variables and analytical logic for the final deliverable.

Main Purpose: Reconstruct the scoreline at the moment of every shot **What It Does:** Builds a goal timeline per match, accumulates home and away goals in event order, then attaches the most recent goal state strictly before each shot and expresses it from the shooting team's point of view **Benefits:** Gives an exact "state before the shot" feature without inspecting the shot itself; a single `merge_asof` replaces the per-shot scan over the goal timeline

```
goal_events = shots.loc[shots["shot_outcome_name"] == "Goal", ["match_id", "index", "team_name"]].copy()
goal_events["home_goal"] = (
    goal_events["team_name"] == goal_events["match_id"].map(matches_flat.set_index("match_id")["home_team_name"])
).astype(int)
goal_events["away_goal"] = (
    goal_events["team_name"] == goal_events["match_id"].map(matches_flat.set_index("match_id")["away_team_name"])
).astype(int)

goal_events = goal_events.sort_values(["match_id", "index"])
goal_events["cum_home_goals"] = goal_events.groupby("match_id")["home_goal"].cumsum()
goal_events["cum_away_goals"] = goal_events.groupby("match_id")["away_goal"].cumsum()

shots_score = (
    pd.merge_asof(
        shots_with_context.assign(_row=np.arange(len(shots_with_context))).sort_values("index"),
        goal_events[["match_id", "index", "cum_home_goals", "cum_away_goals"]].sort_values("index"),
        on="index",
        by="match_id",
        direction="backward",
        allow_exact_matches=False,
    )
    .sort_values("_row")
    .drop(columns="_row")
    .reset_index(drop=True)
)
shots_score["cum_home_goals"] = shots_score["cum_home_goals"].fillna(0)
shots_score["cum_away_goals"] = shots_score["cum_away_goals"].fillna(0)

is_home = shots_score["team_name"] == shots_score["home_team_name"]
shots_score["score_diff_before_shot"] = np.where(
    is_home,
    shots_score["cum_home_goals"] - shots_score["cum_away_goals"],
    shots_score["cum_away_goals"] - shots_score["cum_home_goals"],
).astype(int)

display(shots_score[[
    "match_id", "player_name", "team_name", "index",
    "score_diff_before_shot",
]].head(10))
```

	match_id	player_name	team_name	index	score_diff_before_shot
0	3754217	Aaron Ramsey	Arsenal	96	0
1	3754217	Francesc Fàbregas i Soler	Chelsea	208	0
2	3754217	Alexis Alejandro Sánchez Sánchez	Arsenal	443	0
3	3754217	Diego da Silva Costa	Chelsea	910	0
4	3754217	Theo Walcott	Arsenal	1097	0
5	3754217	Pedro Eliezer Rodríguez Ledesma	Chelsea	1216	0
6	3754217	Aaron Ramsey	Arsenal	1356	0
7	3754217	Kurt Happy Zouma	Chelsea	1433	0
8	3754217	Pedro Eliezer Rodríguez Ledesma	Chelsea	1465	0
9	3754217	Francesc Fàbregas i Soler	Chelsea	1496	0

Main Purpose: Turn match context into multiplicative pressure weights and score the decision **What It Does:** Maps minute, score difference, league position and recent form onto empirically chosen weights; multiplies them into `context_factor`; multiplies `delta` by that factor to obtain `decision_cost` **Benefits:** Expresses four separate contextual ideas on one comparable scale and documents each contribution as its own column, so the final metric stays explainable

```
final_shots = shots_score.copy()

final_shots["minute_factor"] = (
    pd.cut(final_shots["minute"], [-1, 30, 60, 75, np.inf],
           labels=[1.00, 1.05, 1.10, 1.20])
    .astype(float)
)
final_shots["score_factor"] = np.select(
    [final_shots["score_diff_before_shot"] < 0, final_shots["score_diff_before_shot"] == 0],
    [1.12, 1.08],
    default=0.98,
)
final_shots["ranking_factor"] = np.select(
    [final_shots["league_position"] <= 4, final_shots["league_position"] >= 17],
    [1.05, 1.08],
    default=1.00,
)
final_shots["form_factor"] = np.select(
    [final_shots["recent_form_5"] <= 3, final_shots["recent_form_5"] >= 10],
    [1.06, 0.98],
    default=1.00,
)

final_shots["context_factor"] = (
    final_shots["minute_factor"]
    * final_shots["score_factor"]
    * final_shots["ranking_factor"]
    * final_shots["form_factor"]
)
final_shots["decision_cost"] = final_shots["delta"] * final_shots["context_factor"]

print("final_shots shape:", final_shots.shape)
display(final_shots[[
    "player_name", "team_name", "minute", "zone",
    "shot_xg", "xG_pass", "delta",
    "recent_form_5", "league_position", "score_diff_before_shot",
    "context_factor", "decision_cost",
]].head(10))
```

```
final_shots shape: (9908, 35)
```

	player_name	team_name	minute	zone	shot_xg	xG_pass	delta	recent_form_5	league_position	score_diff_before_shot	context_factor	decision_cost
0	Aaron Ramsey	Arsenal	1	Z3_2	0.038832	0.111455	0.072623	7.0	4.0	0	1.13400	0.082355
1	Francesc Fàbregas i Soler	Chelsea	5	Z3_2	0.031541	0.111455	0.079914	4.0	17.0	0	1.16640	0.093211
2	Alexis Alejandro Sánchez Sánchez	Arsenal	11	Z2_1	0.006660	0.085757	0.079097	7.0	4.0	0	1.13400	0.089696
3	Diego da Silva Costa	Chelsea	23	Z3_1	0.022902	0.105515	0.082614	4.0	17.0	0	1.16640	0.096361
4	Theo Walcott	Arsenal	28	Z3_1	0.049741	0.105515	0.055775	7.0	4.0	0	1.13400	0.063249
5	Pedro Eliezer Rodríguez Ledesma	Chelsea	31	Z3_2	0.400975	0.111455	-0.289520	4.0	17.0	0	1.22472	-0.354581
6	Aaron Ramsey	Arsenal	34	Z3_1	0.023207	0.105515	0.082309	7.0	4.0	0	1.19070	0.098005
7	Kurt Happy Zouma	Chelsea	36	Z2_2	0.020538	0.088106	0.067568	4.0	17.0	0	1.22472	0.082752
8	Pedro Eliezer Rodríguez Ledesma	Chelsea	37	Z3_2	0.033888	0.111455	0.077567	4.0	17.0	0	1.22472	0.094998
9	Francesc Fàbregas i Soler	Chelsea	38	Z3_1	0.026702	0.105515	0.078813	4.0	17.0	0	1.22472	0.096524

Main Purpose: Consolidate the enriched shots into one analysis-ready table **What It Does:** Selects the 29 columns used downstream — identity, timing, location, value metrics, context and the final decision cost **Benefits:** Creates a single source of truth for the descriptive analysis and any later modelling

```
final_dataset = final_shots[[
    "season_name", "season_code", "match_id", "match_date",
    "home_team_name", "away_team_name",
    "team_name", "player_name",
    "minute", "second", "index",
    "x", "y", "zone",
    "shot_xg", "xG_pass", "zone_pass_shot_samples", "delta",
    "recent_form_5", "league_position", "cum_points", "cum_gd",
    "score_diff_before_shot",
    "minute_factor", "score_factor", "ranking_factor", "form_factor",
    "context_factor", "decision_cost",
]].copy()

print("final_dataset shape:", final_dataset.shape)
display(final_dataset.head(10))
```

```
final_dataset shape: (9908, 29)
```

	season_name	season_code	match_id	match_date	home_team_name	away_team_name	team_name	player_name	minute	second	...	league_position	cum_points	cum_gd	score_diff_before_shot	minute_factor	score_factor	ranking_factor	form_factor	context_factor	decision_cost
0	2015/2016	1516	3754217	2015-09-19	Chelsea	Arsenal	Arsenal	Aaron Ramsey	1	47	...	4.0	10.0	2.0	0	1.00	1.08	1.05	1.0	1.13400	0.082355
1	2015/2016	1516	3754217	2015-09-19	Chelsea	Arsenal	Chelsea	Francesc Fàbregas i Soler	5	57	...	17.0	4.0	-5.0	0	1.00	1.08	1.08	1.0	1.16640	0.093211
2	2015/2016	1516	3754217	2015-09-19	Chelsea	Arsenal	Arsenal	Alexis Alejandro Sánchez Sánchez	11	17	...	4.0	10.0	2.0	0	1.00	1.08	1.05	1.0	1.13400	0.089696
3	2015/2016	1516	3754217	2015-09-19	Chelsea	Arsenal	Chelsea	Diego da Silva Costa	23	29	...	17.0	4.0	-5.0	0	1.00	1.08	1.08	1.0	1.16640	0.096361
4	2015/2016	1516	3754217	2015-09-19	Chelsea	Arsenal	Arsenal	Theo Walcott	28	36	...	4.0	10.0	2.0	0	1.00	1.08	1.05	1.0	1.13400	0.063249
5	2015/2016	1516	3754217	2015-09-19	Chelsea	Arsenal	Chelsea	Pedro Eliezer Rodríguez Ledesma	31	34	...	17.0	4.0	-5.0	0	1.05	1.08	1.08	1.0	1.22472	-0.354581
6	2015/2016	1516	3754217	2015-09-19	Chelsea	Arsenal	Arsenal	Aaron Ramsey	34	32	...	4.0	10.0	2.0	0	1.05	1.08	1.05	1.0	1.19070	0.098005
7	2015/2016	1516	3754217	2015-09-19	Chelsea	Arsenal	Chelsea	Kurt Happy Zouma	36	34	...	17.0	4.0	-5.0	0	1.05	1.08	1.08	1.0	1.22472	0.082752
8	2015/2016	1516	3754217	2015-09-19	Chelsea	Arsenal	Chelsea	Pedro Eliezer Rodríguez Ledesma	37	16	...	17.0	4.0	-5.0	0	1.05	1.08	1.08	1.0	1.22472	0.094998
9	2015/2016	1516	3754217	2015-09-19	Chelsea	Arsenal	Chelsea	Francesc Fàbregas i Soler	38	27	...	17.0	4.0	-5.0	0	1.05	1.08	1.08	1.0	1.22472	0.096524

Main Purpose: Describe decision costs and rank players **What It Does:** Summarises the value columns; plots the decision-cost distribution and shot xG against xG_pass; aggregates decision cost per player and lists the highest averages among players with at least ten shots **Benefits:** Shows whether the metric is well behaved and which players took the most suboptimal shots relative to the passing alternative available from the same zone

```
print("Summary stats:")
display(final_dataset[["shot_xg", "xG_pass", "delta", "context_factor", "decision_cost"]].describe())

plt.figure(figsize=(8, 5))
final_dataset["decision_cost"].dropna().hist(bins=40)
plt.title("Distribution of Decision Cost")
plt.xlabel("Decision Cost")
plt.ylabel("Frequency")
plt.show()

plt.figure(figsize=(8, 5))
plt.scatter(final_dataset["shot_xg"], final_dataset["xG_pass"], alpha=0.3)
plt.title("xG_shot vs xG_pass")
plt.xlabel("xG_shot")
plt.ylabel("xG_pass")
plt.show()

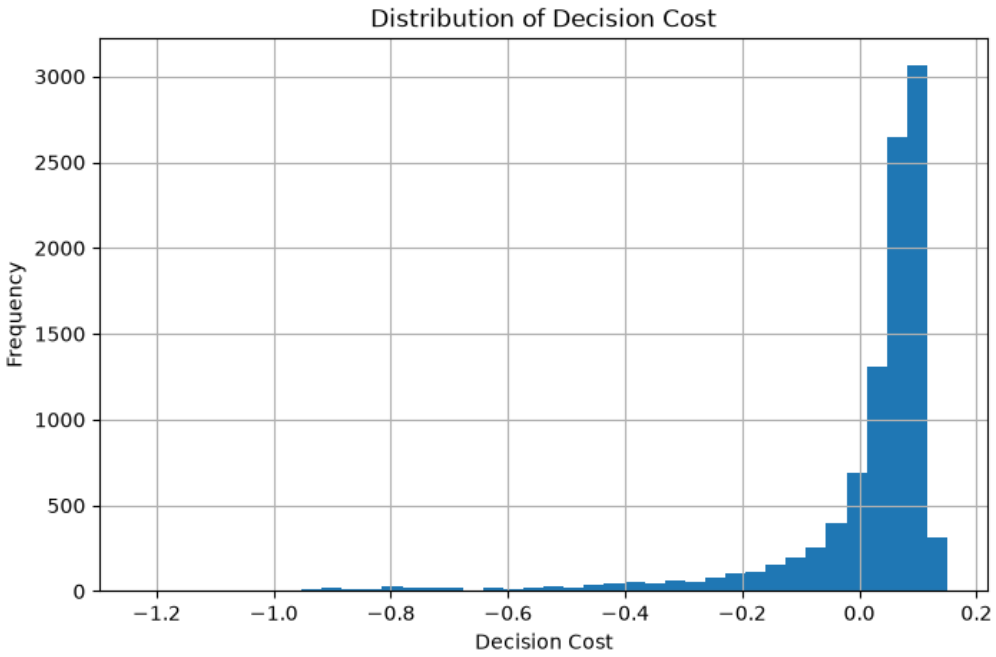
player_summary = (
    final_dataset
    .groupby("player_name", as_index=False)
    .agg(
        num_shots=("decision_cost", "size"),
        avg_decision_cost=("decision_cost", "mean"),
        total_decision_cost=("decision_cost", "sum"),
    )
    .query("num_shots >= 10")
    .sort_values("avg_decision_cost", ascending=False)
)
display(player_summary.head(15))
```

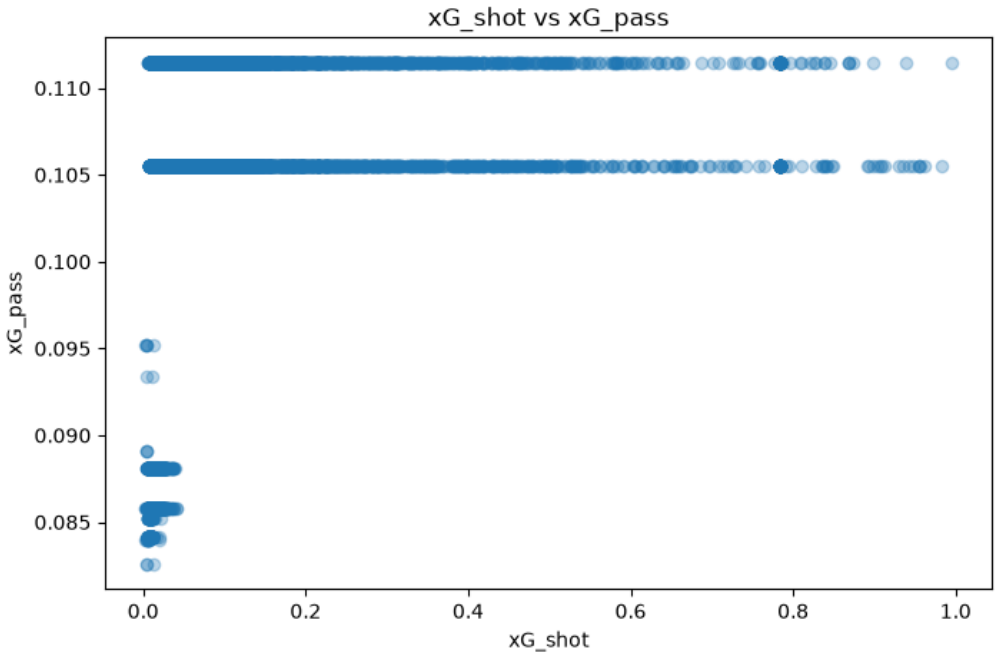
```
Summary stats:
```

	shot_xg	xG_pass	delta	context_factor	decision_cost
count	9908.000000	9907.000000	9907.000000	9908.000000	9907.000000
mean	0.098040	0.106754	0.008704	1.183144	0.010649
std	0.135054	0.006529	0.134169	0.113184	0.158722
min	0.000180	0.082572	-0.883668	0.960400	-1.228337
25%	0.028966	0.105515	0.007197	1.090740	0.008447
50%	0.052942	0.105515	0.055032	1.166886	0.064491
75%	0.101133	0.111455	0.075885	1.246560	0.088805
max	0.995122	0.111455	0.105668	1.538611	0.150203

	player_name	num_shots	avg_decision_cost	total_decision_cost
8	Adlène Guédioura	13	0.099333	1.291333
161	Glenn Whelan	11	0.095043	1.045472
445	Yann Gérard M"Vila	28	0.093402	2.615248
133	Fabian Delph	13	0.090980	1.182735

	player_name	num_shots	avg_decision_cost	total_decision_cost
48	Ben Watson	16	0.089280	1.428485
175	Idrissa Gana Gueye	30	0.088586	2.657582
67	Carlos Alberto Sánchez Moreno	13	0.087262	1.134411
169	Harry Arter	36	0.087175	3.138296
435	Wahbi Khazri	25	0.084970	2.124245
368	Robert Kenedy Nunes do Nascimento	13	0.084447	1.097814
89	Cédric Ricardo Alves Soares	10	0.082920	0.829200
180	Jack Grealish	16	0.081522	1.304360
0	Aaron Cresswell	27	0.080845	2.182823
101	Danny Rose	26	0.080105	2.082726
163	Graham Dorrans	16	0.079480	1.271675





Limitations and Reflection

This project uses publicly available event data to estimate shot decision quality. This design has practical advantages, but also several limitations:

1. StatsBomb open EPL event coverage is limited, so the event-level analysis window is narrower than the full 10-season results window.

2. The passing alternative is not directly observed as an actually available passing lane at the decision moment. Instead, it is estimated from historical pass-to-shot patterns within the same pitch zone.
3. League position and recent form are used as contextual proxies and may not fully represent true psychological or tactical pressure.
4. Event data does not provide full tracking information such as teammate and defender positions.

Despite these limitations, the workflow demonstrates a complete and justified data preparation pipeline: profiling, structuring, enriching, cleaning, feature engineering, and construction of an analysis-ready shot-level dataset.

Data Quality and Missing/Outlier Handling

To ensure that the data preparation pipeline is transparent and justified, we explicitly summarize how missing values and potential outliers were handled.

Missing Values

We inspected key columns used in the final shot-level analytical dataset, including:

- spatial coordinates (`x`, `y`)
- shot value (`shot_xg`)
- team and player identifiers
- engineered contextual features such as `xG_pass`, `recent_form_5`, and `league_position`

Records with missing values in essential columns required for shot-level analysis were removed during earlier preparation steps. In particular:

- shots without valid spatial coordinates or missing `shot_xg` were excluded,
- passes without valid spatial coordinates were excluded,
- joins with results/context data may still lead to some missing contextual features if no suitable historical match state was available.

Outliers and Small-Sample Issues

We also examined potential outliers in engineered variables, especially:

- `shot_xg`
- `xG_pass`
- `decision_cost`

In this project, extreme values are not automatically treated as errors, because some high-value or low-value football events may still be valid observations. Instead, we use profiling and summary statistics to identify unusual values and interpret them carefully.

A more important stability issue is small sample size at the zone level. Since `xG_pass` is estimated from historical pass-to-shot sequences, zones with very few samples may produce unstable averages. We therefore keep the sample count (`zone_pass_shot_samples`) in the final dataset so that low-support estimates can be recognized and discussed explicitly.

Data Quality Note

This project uses multiple public data sources. As a result, some data quality issues arise from:

- limited event-data coverage in publicly available StatsBomb EPL data,
- naming mismatches across sources,
- contextual features that cannot always be perfectly aligned at every event timestamp.

These issues are handled through explicit cleaning, standardization, and cautious interpretation rather than being ignored.

Main Purpose: Check the final dataset for missing values, low-support zones and extreme decisions **What It Does:** Reports missing counts and percentages for the quality-critical columns; describes the numeric columns; isolates shots from zones with fewer than five pass-to-shot samples; lists the ten largest absolute decision costs **Benefits:** Separates genuine findings from artefacts; documents completeness; flags zone estimates that rest on too little evidence to be interpreted confidently

```
QUALITY_COLUMNS = [
    "x", "y", "shot_xg", "xG_pass", "delta",
    "recent_form_5", "league_position",
    "score_diff_before_shot", "context_factor", "decision_cost",
    "zone_pass_shot_samples",
]

print("Final dataset shape:", final_dataset.shape)

print("\nMissing values summary:")
missing_summary = final_dataset[QUALITY_COLUMNS].isna().sum().to_frame("missing_count")
missing_summary["missing_pct"] = (missing_summary["missing_count"] / len(final_dataset) * 100).round(2)
display(missing_summary)

print("\nBasic numeric summary:")
display(final_dataset[QUALITY_COLUMNS].describe())

print("\nRows with low-support zone estimates (zone_pass_shot_samples < 5):")
low_support = final_dataset[final_dataset["zone_pass_shot_samples"] < 5]
print("Count:", len(low_support))
display(low_support[[
    "player_name", "team_name", "zone",
    "shot_xg", "xG_pass", "zone_pass_shot_samples", "decision_cost",
]].head(10))

print("\nPotential extreme decision_cost values (top 10 by absolute value):")
extreme_costs = final_dataset.loc[final_dataset["decision_cost"].abs().nlargest(10).index]
display(extreme_costs[[
    "player_name", "team_name", "zone",
    "shot_xg", "xG_pass", "delta",
    "context_factor", "decision_cost", "zone_pass_shot_samples",
]])
```

```
Final dataset shape: (9908, 29)

Missing values summary:

Basic numeric summary:

Rows with low-support zone estimates (zone_pass_shot_samples < 5):
Count: 0

Potential extreme decision_cost values (top 10 by absolute value):
```

	missing_count	missing_pct
x	0	0.00
y	0	0.00
shot_xg	0	0.00
xG_pass	1	0.01
delta	1	0.01
recent_form_5	489	4.94
league_position	239	2.41
score_diff_before_shot	0	0.00
context_factor	0	0.00
decision_cost	1	0.01
zone_pass_shot_samples	1	0.01

	x	y	shot_xg	xG_pass	delta	recent_form_5	league_position	score_diff_before_shot	context_factor	decision_cost	zone_pass_shot_samples
count	9908.000000	9908.000000	9908.000000	9907.000000	9907.000000	9419.000000	9669.000000	9908.000000	9908.000000	9907.000000	9907.000000
mean	103.622658	39.687303	0.098040	0.106754	0.008704	6.634781	10.237770	-0.025131	1.183144	0.010649	2878.977491
std	8.414518	9.662995	0.135054	0.006529	0.134169	3.311525	5.704091	1.164338	0.113184	0.158722	614.575174
min	35.400000	1.000000	0.000180	0.082572	-0.883668	0.000000	1.000000	-6.000000	0.960400	-1.228337	2596.000000
25%	97.300000	32.400000	0.028966	0.105515	0.007197	4.000000	5.000000	-1.000000	1.090740	0.008447	2596.000000
50%	104.600000	39.900000	0.052942	0.105515	0.055032	7.000000	10.000000	0.000000	1.166886	0.064491	2813.000000
75%	110.500000	46.800000	0.101133	0.111455	0.075885	9.000000	15.000000	0.000000	1.246560	0.088805	2813.000000
max	120.200000	74.800000	0.995122	0.111455	0.105668	15.000000	20.000000	5.000000	1.538611	0.150203	5212.000000

	player_name	team_name	zone	shot_xg	xG_pass	zone_pass_shot_samples	decision_cost

	player_name	team_name	zone	shot_xg	xG_pass	delta	context_factor	decision_cost	zone_pass_shot_samples
867	Jermain Defoe	Sunderland	Z3_1	0.983099	0.105515	-0.877584	1.399680	-1.228337	2813.0
6063	Jermain Defoe	Sunderland	Z3_1	0.907509	0.105515	-0.801993	1.483661	-1.189886	2813.0
2514	Cameron Jerome	Norwich City	Z3_2	0.938525	0.111455	-0.827070	1.344000	-1.111582	2596.0
7017	Jamie Vardy	Leicester City	Z3_2	0.995122	0.111455	-0.883668	1.210104	-1.069330	2596.0
1823	Romelu Lukaku Menama	Everton	Z3_1	0.898100	0.105515	-0.792585	1.344000	-1.065234	2813.0
6698	Diego da Silva Costa	Chelsea	Z3_1	0.961805	0.105515	-0.856290	1.232000	-1.054949	2813.0
1369	Fabio Borini	Sunderland	Z3_1	0.934682	0.105515	-0.829167	1.270080	-1.053108	2813.0
8231	Aleksandar Mitrovi■	Newcastle United	Z3_1	0.783500	0.105515	-0.677985	1.538611	-1.043155	2813.0

	player_name	team_name	zone	shot_xg	xG_pass	delta	context_factor	decision_cost	zone_pass_shot_samples
253	Saido Berahino	West Bromwich Albion	Z3_1	0.954031	0.105515	-0.848516	1.224720	-1.039194	2813.0
3338	Jesús Navas González	Manchester City	Z3_1	0.953960	0.105515	-0.848445	1.190700	-1.010243	2813.0

Source Coverage Note and Scope Clarification

Our project combines multiple public data sources with different coverage characteristics.

Results and Context Window

The match-results data from Football-Data.co.uk was expanded to a 10-season window. This broader time span supports:

- more robust league standing construction,
- recent-form estimation,
- better long-term contextual reasoning.

Event-Level Analysis Window

The core shot-decision analysis depends on event-level data such as:

- shots,
- passes,
- possession sequences,
- shot xG,
- pitch locations.

These variables come from StatsBomb open event data. However, publicly available EPL event coverage in StatsBomb is more limited than the 10-season results window.

Therefore, the final event-level analysis is performed on the EPL seasons that are available in StatsBomb open data, while the broader 10-season results data is used to support contextual preparation and league-based enrichment where applicable.

Justification

This design is still appropriate for the project because:

1. the event-level source is essential for constructing `xG_shot`, `xG_pass`, and possession-based pass-to-shot links,
2. the match-results source provides additional context not directly available in event data,
3. the notebook explicitly documents this source-coverage limitation instead of hiding it.

This reflects a justified source-selection decision under real public-data constraints.

Main Purpose: Summarise the temporal coverage of both sources **What It Does:** Prints the football-data seasons in the results table and the StatsBomb EPL seasons used for event analysis **Benefits:** Provides an audit trail showing which seasons actually contributed to each layer of the pipeline

```
print("Football-data seasons in results:")
print(sorted(results["season_code"].dropna().unique().tolist()))

print("\nStatsBomb EPL seasons used for event-level analysis:")
display(epl[["competition_id", "season_id", "season_name"]].drop_duplicates())
```

```
Football-data seasons in results:
['1415', '1516', '1617', '1718', '1819', '1920', '2021', '2122', '2223', '2324']

StatsBomb EPL seasons used for event-level analysis:
```

	competition_id	season_id	season_name
0	2	27	2015/2016
1	2	44	2003/2004